C++ - Module 03

Inheritance (Héritage)

Résumé :
Ce document contient les exercices du Module 03 des modules C++.

Version : 8.1

Traduction française du sujet. Les termes techniques C++ sont volontairement conservés en anglais.

Sommaire

I Introduction **2**

II Règles générales **3**

III Instructions concernant l’IA **5**

IV Exercice 00 : Aaaaand... OPEN! **7**

V Exercice 01 : Serena, my love! **9**

VI Exercice 02 : Repetitive work **10**

VII Exercice 03 : Now it’s weird! **11**

VIII Soumission et Peer-Evaluation **13**

Chapitre I
Introduction

C++ est un langage de programmation généraliste créé par Bjarne Stroustrup comme une extension du langage de programmation C, ou « C with Classes » (source : Wikipedia).

L'objectif de ces modules est de t'initier à la **programmation orientée objet** (Object-Oriented Programming). Ce sera le point de départ de ton parcours en C++. De nombreux langages sont recommandés pour apprendre la POO, mais nous avons choisi le C++ parce qu'il dérive de ton vieil ami le C. Comme il s'agit d'un langage complexe, et afin de garder les choses simples, ton code devra respecter le standard C++98.

Nous sommes conscients que le C++ moderne est très différent sur de nombreux aspects. Donc, si tu veux devenir une développeuse C++ compétente, c'est à toi d'aller plus loin après le tronc commun de 42 !

Chapitre II
Règles générales

Compilation

- Compile ton code avec `c++` et les flags `-Wall -Wextra -Werror`
- Ton code doit toujours compiler si tu ajoutes le flag `-std=c++98`

Formatage et conventions de nommage

- Les répertoires des exercices seront nommés ainsi : `ex00, ex01, ... , exn`
- Nomme tes fichiers, classes, fonctions, member functions et attributs comme demandé dans les consignes.
- Écris les noms de classes au format **UpperCamelCase**. Les fichiers contenant le code d'une classe seront toujours nommés d'après le nom de la classe. Par exemple : `ClassName.hpp`/`ClassName.h`, `ClassName.cpp` ou `ClassName.tpp`. Ainsi, si tu as un fichier header contenant la définition d'une classe « BrickWall » représentant un mur de briques, il sera nommé `BrickWall.hpp`.
- Sauf indication contraire, chaque message affiché doit se terminer par un caractère de retour à la ligne et être envoyé sur la sortie standard (standard output).
- *Adieu la Norminette !* Aucun style de code n'est imposé dans les modules C++. Tu peux suivre celui que tu préfères. Mais garde à l'esprit qu'un code que tes évaluateurs ne peuvent pas comprendre est un code qu'ils ne peuvent pas noter. Fais de ton mieux pour écrire un code propre et lisible.

Autorisé / Interdit

Tu ne codes plus en C. C'est l'heure du C++ ! Par conséquent :

- Tu peux utiliser presque tout ce que contient la standard library. Ainsi, plutôt que de t'en tenir à ce que tu connais déjà, il serait judicieux d'utiliser autant que possible les versions « à la C++ » des fonctions C dont tu as l'habitude.
- En revanche, tu ne peux utiliser aucune autre bibliothèque externe. Cela signifie que le C++11 (et ses formes dérivées) et les bibliothèques **Boost** sont interdits. Les fonctions suivantes sont également interdites : `*printf()`, `*alloc()` et `free()`. Si tu les utilises, ta note sera 0, et c'est tout.
- Note que, sauf indication explicite contraire, les mots-clés `using namespace <ns_name>` et `friend` sont interdits. Sinon, ta note sera -42.

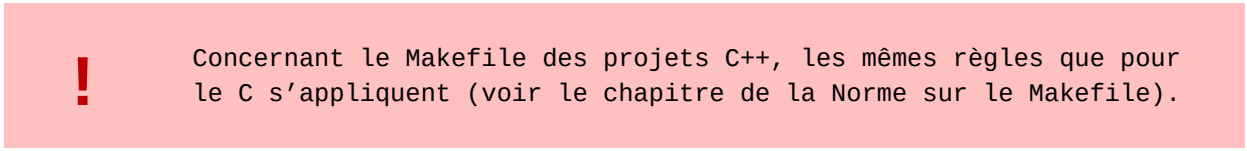
- **Tu n’as le droit d’utiliser la STL que dans les Modules 08 et 09.** Cela signifie : pas de **Containers** (vector/list/map, etc.) et pas d’**Algorithms** (tout ce qui nécessite d’inclure le header `<algorithm>`) d’ici là. Sinon, ta note sera -42.

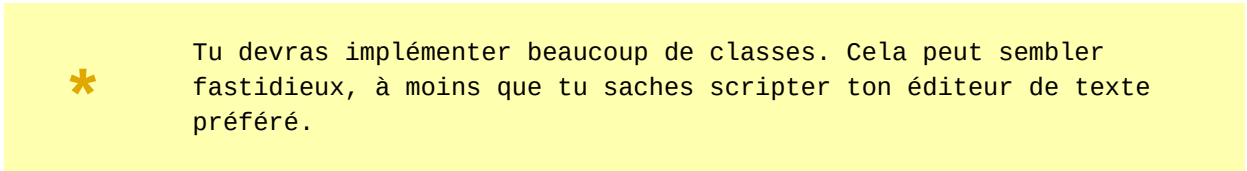
Quelques exigences de conception

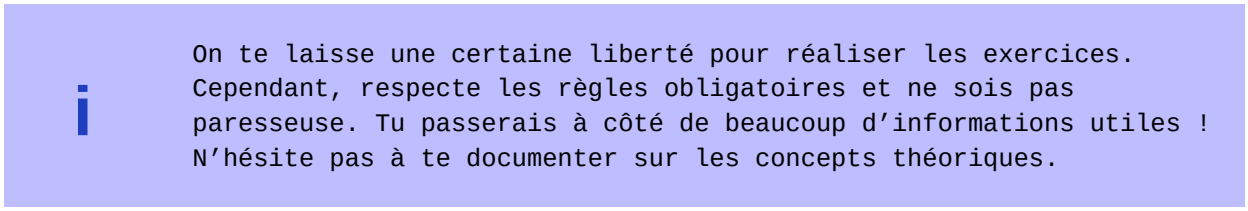
- Les fuites de mémoire existent aussi en C++. Lorsque tu alloues de la mémoire (avec le mot-clé `new`), tu dois éviter les **memory leaks**.
- Du Module 02 au Module 09, tes classes doivent être conçues selon l’**Orthodox Canonical Form, sauf indication explicite contraire**.
- Toute implémentation de fonction placée dans un fichier header (sauf pour les function templates) vaut 0 à l’exercice.
- Tu dois pouvoir utiliser chacun de tes headers indépendamment des autres. Ils doivent donc inclure toutes les dépendances dont ils ont besoin. En revanche, tu dois éviter le problème de la double inclusion en ajoutant des **include guards**. Sinon, ta note sera 0.

À lire

- Tu peux ajouter des fichiers supplémentaires si nécessaire (par exemple pour découper ton code). Comme ces rendus ne sont pas vérifiés par un programme, n’hésite pas à le faire, du moment que tu rends les fichiers obligatoires.
- Parfois, les consignes d’un exercice paraissent courtes, mais les exemples peuvent faire apparaître des exigences qui ne sont pas écrites explicitement dans l’énoncé.
- Lis chaque module en entier avant de commencer ! Vraiment, fais-le.
- Par Odin, par Thor ! Utilise ton cerveau !!!







Chapitre III
Instructions concernant l’IA

Contexte

Ce projet est conçu pour t’aider à découvrir les briques fondamentales de ta formation à 42.

Pour bien ancrer les connaissances et compétences clés, il est essentiel d’adopter une approche réfléchie dans l’utilisation des outils d’IA et du soutien disponible.

Un apprentissage fondamental solide demande un véritable effort intellectuel — à travers le défi, la répétition et les échanges de peer-learning.

Pour un aperçu plus complet de notre position sur l’IA — comme outil d’apprentissage, au sein de la formation 42 et comme attente du marché du travail — consulte la FAQ dédiée sur l’intranet.

Message principal

- Construis des bases solides, sans raccourcis.
- Développe réellement tes compétences techniques et tes « power skills ».
- Vis un vrai peer-learning, commence à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d’apprentissage est plus important que le résultat.
- Découvre les risques associés à l’IA, et développe des pratiques de contrôle et des contre-mesures efficaces pour éviter les pièges courants.

Règles de l’apprenant :

- Tu dois appliquer ton raisonnement aux tâches qui te sont confiées, surtout avant de te tourner vers l’IA.
- Tu ne dois pas demander de réponses directes à l’IA.
- Tu dois te renseigner sur l’approche globale de 42 concernant l’IA.

Résultats de la phase :

Au cours de cette phase fondamentale, tu obtiendras les résultats suivants :

- Acquérir de solides bases techniques et en programmation.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre tes projets. Mais tu es ici pour apprendre, pas pour prouver que l'IA a appris. Ne perds pas ton temps (ni le nôtre) juste pour démontrer que l'IA sait résoudre le problème donné.
- Apprendre à 42, ce n'est pas connaître la réponse — c'est développer la capacité d'en trouver une. L'IA te donne directement la réponse, mais cela t'empêche de construire ton propre raisonnement. Or raisonner demande du temps, des efforts, et implique d'échouer. Le chemin vers la réussite n'est pas censé être facile.
- Garde en tête que pendant les examens, l'IA n'est pas disponible — pas d'internet, pas de smartphone, etc. Tu verras vite si tu t'es trop appuyée sur l'IA dans ton apprentissage.
- Le peer-learning t'expose à des idées et à des approches différentes, améliore tes compétences interpersonnelles et ta capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne sois pas timide — parle, pose des questions et apprenez ensemble !
- Oui, l'IA fera partie du cursus — à la fois comme outil d'apprentissage et comme sujet à part entière. Tu auras même l'occasion de construire ton propre logiciel d'IA. Pour en savoir plus sur notre approche progressive (« crescendo »), consulte la documentation disponible sur l'intranet.

Bonne pratique :

> Je suis bloquée sur un nouveau concept. Je demande à quelqu'un à côté de moi comment il ou elle s'y est pris. Nous discutons pendant 10 minutes — et soudain, ça fait tilt. Je comprends.

Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui a l'air correct. Pendant la peer-evaluation, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis de nouveau bloquée. J'échoue.

Chapitre IV
Exercice 00 : Aaaaand... OPEN!

Exercice : 00
Aaaaand... OPEN!
Répertoire : *ex00/*
Fichiers à rendre : `Makefile, main.cpp, ClapTrap.{h, hpp}, ClapTrap.cpp`
Fonctions interdites : Aucune

Tout d’abord, tu dois implémenter une classe ! Quelle originalité !

Elle s’appellera **ClapTrap** et aura les attributs privés suivants, initialisés avec les valeurs indiquées entre parenthèses :

- Name, qui est passé en paramètre au constructor
- Hit points (10), représentant la santé du ClapTrap
- Energy points (10)
- Attack damage (0)

Ajoute les member functions publiques suivantes afin que le ClapTrap se comporte de façon plus réaliste :

- `void attack(const std::string& target);`
- `void takeDamage(unsigned int amount);`
- `void beRepaired(unsigned int amount);`

Quand ClapTrap attaque, sa cible perd <attack damage> hit points. Quand ClapTrap se répare, il regagne <amount> hit points. Attaquer et se réparer coûtent chacun 1 energy point. Bien sûr, ClapTrap ne peut rien faire s’il n’a plus de hit points ou d’energy points. Cependant, comme ces exercices servent d’introduction, les instances de ClapTrap ne doivent pas interagir directement entre elles, et les paramètres ne feront pas référence à une autre instance de ClapTrap.

Dans toutes ces member functions, tu dois afficher un message pour décrire ce qui se passe. Par exemple, la fonction `attack()` pourra afficher quelque chose comme (bien sûr, sans les chevrons) :

```
ClapTrap <name> attacks <target>, causing <damage> points of damage!
```

Les constructors et le destructor doivent aussi afficher un message, afin que tes évaluateurs (peer-evaluators) puissent facilement voir qu’ils ont bien été appelés.

Implémente et rends tes propres tests pour t’assurer que ton code fonctionne comme prévu.

Chapitre V
Exercice 01 : Serena, my love!

Exercice : 01
Serena, my love!
Répertoire : *ex01/*
Fichiers à rendre : `Fichiers de l'exercice précédent + ScavTrap.{h, hpp}, ScavTrap.cpp`
Fonctions interdites : Aucune

Parce qu'on n'aura jamais assez de ClapTraps, tu vas maintenant créer un robot dérivé. Il s'appellera **ScavTrap** et **héritera des constructors et du destructor** de ClapTrap. Cependant, ses constructors, son destructor et `attack()` afficheront des messages différents. Après tout, les ClapTraps ont conscience de leur individualité.

Note que l'enchaînement correct de construction/destruction doit être démontré dans tes tests. Quand un ScavTrap est créé, le programme commence par construire un ClapTrap. La destruction se fait dans l'ordre inverse. Pourquoi ?

ScavTrap utilisera les attributs de ClapTrap (**mets ClapTrap à jour en conséquence**) et doit les initialiser à :

- Name, qui est passé en paramètre au constructor
- Hit points (100), représentant la santé du ClapTrap
- Energy points (50)
- Attack damage (20)

ScavTrap aura aussi sa propre capacité spéciale :

```
void guardGate();
```

Cette member function affichera un message indiquant que ScavTrap est maintenant en mode Gate keeper.

N'oublie pas d'ajouter davantage de tests à ton programme.

Chapitre VI
Exercice 02 : Repetitive work

Exercice : 02
Repetitive work
Répertoire : *ex02/*
Fichiers à rendre : `Fichiers des exercices précédents + FragTrap.{h, hpp}, FragTrap.cpp`
Fonctions interdites : Aucune

Fabriquer des ClapTraps commence probablement à te taper sur les nerfs.

Maintenant, implémente une classe **FragTrap** qui hérite de ClapTrap. Elle est très proche de ScavTrap. Cependant, ses messages de construction et de destruction doivent être différents. L'enchaînement correct de construction/destruction doit être démontré dans tes tests. Quand un FragTrap est créé, le programme commence par construire un ClapTrap. La destruction se fait dans l'ordre inverse.

Pareil pour les attributs, mais avec des valeurs différentes cette fois :

- Name, qui est passé en paramètre au constructor
- Hit points (100), représentant la santé du ClapTrap
- Energy points (100)
- Attack damage (30)

FragTrap a lui aussi une capacité spéciale :

```
void highFivesGuys(void);
```

Cette member function affiche sur la sortie standard une demande de high five positive.

Encore une fois, ajoute davantage de tests à ton programme.

Chapitre VII
Exercice 03 : Now it’s weird!

Exercice : 03
Now it’s weird!
Répertoire : *ex03/*
Fichiers à rendre : `Fichiers des exercices précédents + DiamondTrap.{h, hpp}, DiamondTrap.cpp`
Fonctions interdites : Aucune

Dans cet exercice, tu vas créer un monstre : un ClapTrap mi-FragTrap, mi-ScavTrap. Il s’appellera **DiamondTrap**, et il héritera à la fois de FragTrap ET de ScavTrap. C’est tellement risqué !

La classe DiamondTrap aura un attribut privé nommé `name`. Cet attribut doit avoir exactement le même nom de variable que celui de la classe de base ClapTrap (sans faire référence au nom du robot).

Pour être plus clair, voici deux exemples :
Si la variable de ClapTrap s’appelle `name`, donne à la variable de DiamondTrap le nom `name`.
Si la variable de ClapTrap s’appelle `_name`, donne à la variable de DiamondTrap le nom `_name`.

Ses attributs et member functions seront hérités de ses classes parentes :

- Name, qui est passé en paramètre à un constructor
- `ClapTrap::name` (paramètre du constructor + suffixe "`_clap_name`")
- Hit points (FragTrap)
- Energy points (ScavTrap)
- Attack damage (FragTrap)
- `attack()` (ScavTrap)

En plus des fonctions spéciales des deux classes parentes, DiamondTrap aura sa propre capacité spéciale :

```
void whoAmI();
```

Cette member function affichera à la fois son nom et son nom de ClapTrap.

Bien sûr, l’instance de ClapTrap de DiamondTrap sera créée une fois, et une seule fois. Oui, il y a une astuce.

Encore une fois, ajoute davantage de tests à ton programme.

Connais-tu les flags de compilation -Wshadow et -Wno-shadow ?

Tu peux valider ce module sans avoir terminé l’exercice 03.

Chapitre VIII
Soumission et Peer-Evaluation

Rends ton travail dans ton dépôt `Git` comme d'habitude. Seul le travail présent dans ton dépôt sera évalué pendant la soutenance. N'hésite pas à revérifier les noms de tes dossiers et fichiers pour t'assurer qu'ils sont corrects.

Pendant l'évaluation, une brève **modification du projet** pourra occasionnellement être demandée. Il peut s'agir d'un changement mineur de comportement, de quelques lignes de code à écrire ou à réécrire, ou d'une fonctionnalité facile à ajouter.

Bien que cette étape puisse **ne pas s'appliquer à tous les projets**, tu dois y être préparée si elle est mentionnée dans les consignes d'évaluation.

Cette étape sert à vérifier ta compréhension réelle d'une partie précise du projet. La modification peut être réalisée dans n'importe quel environnement de développement de ton choix (par exemple ton installation habituelle), et elle doit être faisable en quelques minutes — sauf si un délai précis est défini dans le cadre de l'évaluation.
Tu peux, par exemple, devoir faire une petite mise à jour d'une fonction ou d'un script, modifier un affichage, ou adapter une structure de données pour stocker de nouvelles informations, etc.

Les détails (périmètre, cible, etc.) seront précisés dans les **consignes d'évaluation** et peuvent varier d'une évaluation à l'autre pour un même projet.